

KMP – Tìm kiếm chuỗi nhanh và dễ hiểu

1. Bài toán tìm kiếm chuỗi

Cho hai chuỗi:

- Chuỗi văn bản **S**, có độ dài **n**
- Chuỗi mẫu **P**, có độ dài **m**

Yêu cầu:

Tìm tất cả vị trí mà chuỗi **P** xuất hiện trong chuỗi **S**.

Ví dụ:

```
S = "ababcbcabababd"
P = "ababd"
```

Ta cần tìm xem "**ababd**" xuất hiện bắt đầu từ vị trí nào trong **S**.

2. Cách làm đơn giản

Ta thử đặt chuỗi **P** tại từng vị trí của **S**.

Ví dụ:

```
S = a b a b c a b c a b a b a b d
P = a b a b d
```

Thử từ vị trí 0:

```
S = a b a b c ...
P = a b a b d
      ^
```

Ta khớp được:

```
a = a
b = b
a = a
b = b
```

Nhưng:

```
c != d
```

Cách thông thường sẽ dịch chuỗi **P** sang phải một ô rồi so sánh lại từ đầu.

Nhược điểm:

Ta đã biết khá nhiều ký tự phía trước giống nhau, nhưng lại bỏ toàn bộ thông tin đó và so sánh lại.

Độ phức tạp tệ nhất:

```
O(n × m)
```

Ví dụ dễ gây chậm:

```
S = "aaaaaaaaaaaaaaaaab"  
P = "aaaaaaaab"
```

Ở rất nhiều vị trí, ta so sánh được gần hết chuỗi rồi mới phát hiện sai.

3. Ý tưởng chính của KMP

KMP là viết tắt của:

```
Knuth – Morris – Pratt
```

Ý tưởng quan trọng nhất:

■ Khi xảy ra sai khác, không cần đưa con trỏ của chuỗi mẫu về đầu hoàn toàn.

Ta tận dụng phần đã khớp trước đó.

Giả sử ta đang so sánh:

```
S = ... a b a b a c ...  
P =   a b a b a b  
           ^
```

Ta đã khớp được:

```
ababa
```

Sau đó gặp sai khác:

```
S: ababa c
P: ababa b
      ^
```

Trong chuỗi "ababa":

- Tiền tố "aba"
- Hậu tố "aba"

giống nhau.

Ta có:

```
ababa
^^^
    ^^^
```

Do đó không cần quay về đầu.

Ta có thể coi phần "aba" ở cuối đã khớp với phần "aba" ở đầu.

4. Prefix và suffix

4.1. Prefix là gì?

Prefix là tiền tố của chuỗi.

Ví dụ:

```
s = "abcd"
```

Các prefix:

```
""
"a"
"ab"
```

```
"abc"  
"abcd"
```

Nếu chỉ xét prefix không phải toàn bộ chuỗi:

```
""  
"a"  
"ab"  
"abc"
```

4.2. Suffix là gì?

Suffix là hậu tố của chuỗi.

Với:

```
s = "abcd"
```

Các suffix:

```
""  
"d"  
"cd"  
"bcd"  
"abcd"
```

4.3. Border là gì?

Một chuỗi vừa là prefix, vừa là suffix được gọi là border.

Ví dụ:

```
s = "ababa"
```

Các prefix:

```
"a"  
"ab"  
"aba"  
"abab"
```

Các suffix:

```
"a"  
"ba"  
"aba"  
"baba"
```

Các border:

```
"a"  
"aba"
```

Border dài nhất là:

```
"aba"
```

Độ dài bằng:

```
3
```

5. Mảng prefix function

KMP thường sử dụng mảng:

```
pi[i]
```

Ý nghĩa:

```
pi[i] = độ dài prefix dài nhất của s[0...i]  
        đồng thời cũng là suffix của s[0...i]  
        nhưng không được bằng toàn bộ chuỗi
```

Nói đơn giản:

Với đoạn từ đầu chuỗi tới vị trí i , có bao nhiêu ký tự ở cuối giống với các ký tự ở đầu?

6. Ví dụ tính mảng pi

Xét chuỗi:

```
s = "ababa"
```

Đánh số:

i	0	1	2	3	4
s[i]	a	b	a	b	a

$i = 0$

Đoạn đang xét:

```
"a"
```

Không có prefix đúng nào vừa là suffix.

```
pi[0] = 0
```

$i = 1$

Đoạn:

```
"ab"
```

Prefix:

```
"a"
```

Suffix:

```
"b"
```

Không giống nhau.

```
pi[1] = 0
```

i = 2

Đoạn:

```
"aba"
```

Prefix:

```
"a"
```

Suffix:

```
"a"
```

Giống nhau.

```
pi[2] = 1
```

i = 3

Đoạn:

```
"abab"
```

Prefix dài nhất bằng suffix:

```
"ab"
```

```
pi[3] = 2
```

i = 4

Đoạn:

```
"ababa"
```

Prefix dài nhất bằng suffix:

```
"aba"
```

```
pi[4] = 3
```

Kết quả:

s	a	b	a	b	a
pi	0	0	1	2	3

7. Cách tính pi hiệu quả

Code:

```
vector<int> prefixFunction(string s) {  
    int n = s.size();  
  
    vector<int> pi(n);  
  
    for(int i = 1; i < n; i++) {  
        int j = pi[i - 1];  
  
        while(j > 0 && s[i] != s[j]) {  
            j = pi[j - 1];  
        }  
  
        if(s[i] == s[j]) {  
            j++;  
        }  
    }  
}
```



```
        pi[i] = j;
    }

    return pi;
}
```

Độ phức tạp:

$O(n)$

8. Giải thích từng dòng

```
for(int i = 1; i < n; i++)
```

Ta tính `pi[i]`.

Không cần tính `pi[0]` vì:

```
pi[0] = 0
```

```
int j = pi[i - 1];
```

Ta thử mở rộng border tốt nhất của đoạn trước.

Ví dụ:

```
s = "ababa..."
```

Giả sử:

```
pi[i - 1] = 3
```

Nghĩa là đoạn trước đang có prefix và suffix giống nhau với độ dài 3.

Ta thử kiểm tra:

```
s[i] == s[3]
```

Nếu giống thì border tăng thêm một ký tự.

```
while(j > 0 && s[i] != s[j])
```

Nếu ký tự mới không khớp, border hiện tại không dùng được.

Ta thử border ngắn hơn.

```
j = pi[j - 1];
```

Dòng này là trái tim của KMP.

Thay vì:

```
j--;
```

ta nhảy thẳng tới border dài nhất tiếp theo.

9. Tại sao lại là $pi[j - 1]$?

Giả sử:

```
j = 5
```

Nghĩa là ta đang có một border độ dài 5.

Nếu:

```
s[i] != s[5]
```

thì border độ dài 5 thất bại.

Ta cần tìm một border ngắn hơn của chính đoạn border đó.

Đoạn border có vị trí:

```
0 đến j - 1
```

Do đó border tốt nhất tiếp theo là:

```
pi[j - 1]
```

10. Ví dụ mô phỏng chi tiết

Xét:

```
s = "ababaca"
```

Ta tính:

```
i      0 1 2 3 4 5 6
s[i]   a b a b a c a
```

i = 1

```
j = pi[0] = 0
```

So sánh:

```
s[1] = 'b'
s[0] = 'a'
```

Khác nhau.

```
pi[1] = 0
```

i = 2

```
j = pi[1] = 0
```

```
s[2] = 'a'  
s[0] = 'a'
```

Giống nhau:

```
j = 1
```

```
pi[2] = 1
```

i = 3

```
j = pi[2] = 1
```

```
s[3] = 'b'  
s[1] = 'b'
```

Giống:

```
j = 2
```

```
pi[3] = 2
```

i = 4

```
j = pi[3] = 2
```

```
s[4] = 'a'
```

```
s[2] = 'a'
```

Giống:

```
j = 3
```

```
pi[4] = 3
```

i = 5

```
j = pi[4] = 3
```

So sánh:

```
s[5] = 'c'  
s[3] = 'b'
```

Khác.

Ta lùi:

```
j = pi[2] = 1
```

So sánh:

```
s[5] = 'c'  
s[1] = 'b'
```

Vẫn khác.

Lùi tiếp:

```
j = pi[0] = 0
```

So sánh:

```
s[5] = 'c'  
s[0] = 'a'
```

Khác.

```
pi[5] = 0
```

i = 6

```
j = pi[5] = 0
```

```
s[6] = 'a'  
s[0] = 'a'
```

Giống:

```
pi[6] = 1
```

Kết quả:

```
s      a b a b a c a  
pi      0 0 1 2 3 0 1
```

11. Dùng KMP để tìm chuỗi

Có hai cách phổ biến.

Cách 1: Ghép chuỗi

Ta tạo:

```
T = P + "#" + S
```

Trong đó:

- **P** là pattern
- **S** là text
- **#** là ký tự không xuất hiện trong hai chuỗi

Ví dụ:

```
P = "aba"
S = "abacaba"
```

Ta tạo:

```
T = "aba#abacaba"
```

Tính mảng **pi** của **T**.

Nếu tại vị trí nào:

```
pi[i] = độ dài P
```

thì pattern xuất hiện.

Code

```
#include<bits/stdc++.h>
using namespace std;

vector<int> prefixFunction(string s) {
    int n = s.size();

    vector<int> pi(n);

    for(int i = 1; i < n; i++) {
        int j = pi[i - 1];

        while(j > 0 && s[i] != s[j]) {
            j = pi[j - 1];
        }

        if(s[i] == s[j]) {
            j++;
        }

        pi[i] = j;
    }
}
```

```
    }

    return pi;
}

int main() {
    string text, pattern;

    cin >> text >> pattern;

    string s = pattern + "#" + text;

    vector<int> pi = prefixFunction(s);

    int m = pattern.size();

    for(int i = m + 1; i < (int)s.size(); i++) {
        if(pi[i] == m) {
            int pos = i - 2 * m;

            cout << pos << '\n';
        }
    }
}
```

12. Công thức vị trí

Ta có:

```
s = pattern + "#" + text
```

Giả sử:

```
pi[i] = m
```

Pattern kết thúc tại vị trí i .

Vị trí bắt đầu trong chuỗi ghép:

```
i - m + 1
```

Phần text bắt đầu tại:

$$m + 1$$

Do đó vị trí trong text:

$$i - m + 1 - (m + 1)$$

Rút gọn:

$$i - 2m$$

13. Ví dụ

```
text = "ababa"  
pattern = "aba"
```

Các lần xuất hiện:

```
ababa  
aba
```

Bắt đầu tại:

```
0
```

Và:

```
ababa  
aba
```

Bắt đầu tại:

```
2
```

Kết quả:

0
2

KMP vẫn tìm được hai lần xuất hiện chồng lấn.

14. Cách 2: Không ghép chuỗi

Ta tính mảng `pi` cho pattern.

Sau đó duyệt text.

```
vector<int> KMP(string text, string pattern) {  
    vector<int> pi = prefixFunction(pattern);  
  
    vector<int> positions;  
  
    int n = text.size();  
    int m = pattern.size();  
  
    int j = 0;  
  
    for(int i = 0; i < n; i++) {  
        while(j > 0 && text[i] != pattern[j]) {  
            j = pi[j - 1];  
        }  
  
        if(text[i] == pattern[j]) {  
            j++;  
        }  
  
        if(j == m) {  
            positions.push_back(i - m + 1);  
  
            j = pi[j - 1];  
        }  
    }  
  
    return positions;  
}
```

15. Giải thích biến j

Trong lúc duyệt text:

```
j = số ký tự đầu tiên của pattern đang khớp
```

Ví dụ:

```
pattern = "abcde"
```

Nếu:

```
j = 3
```

thì ta đang khớp được:

```
"abc"
```

Ký tự tiếp theo cần kiểm tra:

```
pattern[3] = 'd'
```

16. Khi tìm thấy pattern

```
if(j == m)
```

Nghĩa là:

```
m ký tự đã khớp
```

Pattern kết thúc tại vị trí:

```
i
```

Vị trí bắt đầu:

```
i - m + 1
```

Sau đó:

```
j = pi[j - 1];
```

để tiếp tục tìm các lần xuất hiện có thể chồng lấn.

Ví dụ:

```
text = "aaaaa"  
pattern = "aaa"
```

Các lần xuất hiện:

```
0  
1  
2
```

17. Độ phức tạp của KMP

Tính mảng **pi**:

```
O(m)
```

Duyệt text:

```
O(n)
```

Tổng:

```
O(n + m)
```

Bộ nhớ:

$O(n)$

18. Tại sao vòng while không làm thuật toán thành $O(n^2)$?

Nhìn đoạn:

```
while(j > 0 && s[i] != s[j]) {  
    j = pi[j - 1];  
}
```

Có vẻ một vị trí i có thể lặp rất nhiều lần.

Nhưng xét toàn bộ thuật toán:

- j chỉ tăng tối đa n lần
- Mỗi lần chạy `while`, j giảm
- Tổng số lần giảm không thể lớn hơn tổng số lần tăng quá nhiều

Do đó tổng độ phức tạp vẫn là:

$O(n)$

Đây gọi là phân tích độ phức tạp khấu hao.

19. Ứng dụng 1: Tìm tất cả vị trí xuất hiện

Bài toán:

Cho chuỗi S và P .

In ra tất cả vị trí mà P xuất hiện trong S .

Code:

```
#include<bits/stdc++.h>  
using namespace std;  
  
vector<int> prefixFunction(string s) {  
    int n = s.size();  
  
    vector<int> pi(n);
```

```

for(int i = 1; i < n; i++) {
    int j = pi[i - 1];

    while(j > 0 && s[i] != s[j]) {
        j = pi[j - 1];
    }

    if(s[i] == s[j]) {
        j++;
    }

    pi[i] = j;
}

return pi;
}

int main() {
    string s, p;

    cin >> s >> p;

    vector<int> pi = prefixFunction(p);

    int j = 0;

    for(int i = 0; i < (int)s.size(); i++) {
        while(j > 0 && s[i] != p[j]) {
            j = pi[j - 1];
        }

        if(s[i] == p[j]) {
            j++;
        }

        if(j == (int)p.size()) {
            cout << i - p.size() + 1 << ' ';

            j = pi[j - 1];
        }
    }
}

```

20. Ứng dụng 2: Đếm số lần xuất hiện

Thay vì lưu vị trí, ta tăng biến đếm.

```

int countOccurrences(string text, string pattern) {
    vector<int> pi = prefixFunction(pattern);

```

```
int cnt = 0;

int j = 0;

for(int i = 0; i < (int)text.size(); i++) {
    while(j > 0 && text[i] != pattern[j]) {
        j = pi[j - 1];
    }

    if(text[i] == pattern[j]) {
        j++;
    }

    if(j == (int)pattern.size()) {
        cnt++;

        j = pi[j - 1];
    }
}

return cnt;
}
```

21. Ứng dụng 3: Tìm chu kỳ nhỏ nhất của chuỗi

Đây là ứng dụng rất phổ biến.

Ví dụ:

```
s = "abccabccabc"
```

Ta nhận thấy:

```
s = "abc" lặp lại 3 lần
```

Chu kỳ nhỏ nhất có độ dài:

```
3
```

Công thức

Gọi:

```
n = độ dài chuỗi
```

Đặt:

```
k = n - pi[n - 1]
```

Nếu:

```
n % k == 0
```

thì chu kỳ nhỏ nhất là:

```
k
```

Ngược lại:

```
n
```

Ví dụ

```
s = "abccabccabc"
```

Ta có:

```
n = 9
```

Border dài nhất:

```
"abccabc"
```

Độ dài:

```
pi[n - 1] = 6
```


Do đó:

$$k = 9 - 6 = 3$$

Vì:

$$9 \% 3 = 0$$

Chu kỳ nhỏ nhất bằng:

3

Code

```
int minimumPeriod(string s) {  
    vector<int> pi = prefixFunction(s);  
  
    int n = s.size();  
  
    int k = n - pi[n - 1];  
  
    if(n % k == 0) {  
        return k;  
    }  
  
    return n;  
}
```

22. Tại sao công thức chu kỳ là $n - \text{pi}[n - 1]$?

Giả sử:

```
s = "abccabccab"
```

Prefix dài nhất bằng suffix:

```
prefix = "abccab"
```

```
suffix = "abcabc"
```

Phần bị lệch giữa prefix và suffix:

```
n - border
```

là:

```
9 - 6 = 3
```

Chính là độ dài một block lặp:

```
"abc"
```

23. Ứng dụng 4: Kiểm tra chuỗi có được tạo bởi việc lặp một chuỗi nhỏ hơn không

Ví dụ:

```
"ababab"
```

Được tạo bởi:

```
"ab" lặp 3 lần
```

Code

```
bool isRepeatedString(string s) {  
    int n = s.size();  
  
    vector<int> pi = prefixFunction(s);  
  
    int k = n - pi[n - 1];
```

```
    return k < n && n % k == 0;  
}
```

24. Ứng dụng 5: Tìm border dài nhất

Ta chỉ cần:

```
pi[n - 1]
```

Ví dụ:

```
s = "abcab"
```

Prefix dài nhất bằng suffix:

```
"ab"
```

Do đó:

```
pi[4] = 2
```

25. Ứng dụng 6: Liệt kê tất cả border

Ví dụ:

```
s = "ababa"
```

Các border:

```
"a"  
"aba"
```

Ta bắt đầu:

```
int k = pi[n - 1];
```

Sau đó:

```
k = pi[k - 1];
```

cho đến khi:

```
k = 0
```

Code

```
vector<int> getAllBorders(string s) {  
    int n = s.size();  
  
    vector<int> pi = prefixFunction(s);  
  
    vector<int> borders;  
  
    int k = pi[n - 1];  
  
    while(k > 0) {  
        borders.push_back(k);  
  
        k = pi[k - 1];  
    }  
  
    reverse(borders.begin(), borders.end());  
  
    return borders;  
}
```

Ví dụ

```
s = "aaaa"
```

Ta có:

```
pi = 0 1 2 3
```

Bắt đầu:

```
k = 3
```

Ta lấy:

```
3
```

Sau đó:

```
k = pi[2] = 2
```

Lấy:

```
2
```

Sau đó:

```
k = pi[1] = 1
```

Lấy:

```
1
```

Các độ dài border:

```
1  
2  
3
```

26. Ứng dụng 7: Đếm số lần mỗi prefix xuất hiện trong

chuỗi

Bài toán:

Với mỗi độ dài k , cần biết:

```
s[0...k-1]
```

xuất hiện bao nhiêu lần trong s .

Ví dụ:

```
s = "ababab"
```

Prefix:

```
"a"
```

xuất hiện:

```
3 lần
```

Prefix:

```
"ab"
```

xuất hiện:

```
3 lần
```

Prefix:

```
"abab"
```

xuất hiện:

2 lần

Ý tưởng

Mỗi giá trị:

```
pi[i] = k
```

nghĩa là prefix độ dài k xuất hiện kết thúc tại vị trí i .

Ta đếm:

```
cnt[pi[i]]++;
```

Sau đó cần đẩy số lần xuất hiện từ prefix dài xuống các border ngắn hơn.

Code

```
vector<int> countPrefixOccurrences(string s) {  
    int n = s.size();  
  
    vector<int> pi = prefixFunction(s);  
  
    vector<int> cnt(n + 1);  
  
    for(int i = 0; i < n; i++) {  
        cnt[pi[i]]++;  
    }  
  
    for(int len = n; len > 0; len--) {  
        cnt[pi[len - 1]] += cnt[len];  
    }  
  
    for(int len = 0; len <= n; len++) {  
        cnt[len]++;  
    }  
  
    return cnt;  
}
```

27. Vì sao phải cộng 1?

Mỗi prefix luôn xuất hiện ít nhất một lần:

```
ngay tại đầu chuỗi
```

Ví dụ:

```
s = "abc"
```

Prefix:

```
"abc"
```

xuất hiện một lần.

Nhưng không có `pi[i] = 3`.

Do đó cuối cùng phải:

```
cnt[len]++;
```

28. Ứng dụng 8: Tìm prefix dài nhất xuất hiện ít nhất K lần

Ta sử dụng mảng đếm số lần xuất hiện của prefix.

Ví dụ:

```
s = "ababab"  
K = 3
```

Các prefix:


```
"a" xuất hiện 3 lần  
"ab" xuất hiện 3 lần  
"aba" xuất hiện 2 lần
```

Prefix dài nhất xuất hiện ít nhất 3 lần là:

```
"ab"
```

Code

```
int longestPrefixAtLeastKTimes(string s, int k) {  
    vector<int> cnt = countPrefixOccurrences(s);  
  
    for(int len = s.size(); len >= 1; len--) {  
        if(cnt[len] >= k) {  
            return len;  
        }  
    }  
  
    return 0;  
}
```

29. Ứng dụng 9: Bài toán Just a legend

Đây là dạng rất nổi tiếng.

Yêu cầu:

Tìm chuỗi dài nhất:

- Là prefix
- Là suffix
- Đồng thời xuất hiện ở giữa chuỗi

Ví dụ:

```
s = "fixprefixsuffix"
```

Ta cần tìm một chuỗi vừa ở đầu, cuối và xuất hiện thêm ở giữa.

Ý tưởng

Độ dài border dài nhất:

```
k = pi[n - 1];
```

Kiểm tra xem border độ dài k có xuất hiện trước vị trí cuối không.

Nếu không:

```
k = pi[k - 1];
```

Code

```
string longestPrefixSuffixMiddle(string s) {
    int n = s.size();

    vector<int> pi = prefixFunction(s);

    vector<bool> appeared(n + 1);

    for(int i = 0; i < n - 1; i++) {
        appeared[pi[i]] = true;
    }

    int k = pi[n - 1];

    while(k > 0 && !appeared[k]) {
        k = pi[k - 1];
    }

    if(k == 0) {
        return "";
    }

    return s.substr(0, k);
}
```

30. Ứng dụng 10: Ghép hai chuỗi với phần chồng lấn lớn nhất

Bài toán:

Cho hai chuỗi:

```
A  
B
```

Tìm độ dài lớn nhất sao cho:

```
suffix của A = prefix của B
```

Ví dụ:

```
A = "abcde"  
B = "defgh"
```

Phần chồng:

```
"de"
```

Độ dài:

```
2
```

Ta có thể ghép:

```
abcdefgh
```

thay vì:

```
abcdedefgh
```

Cách làm

Tạo:

```
B + "#" + A
```

Giá trị:

```
pi.back()
```

là độ dài chồng lấn lớn nhất.

Code

```
int maximumOverlap(string a, string b) {  
    string s = b + "#" + a;  
  
    vector<int> pi = prefixFunction(s);  
  
    return pi.back();  
}
```

31. Ứng dụng 11: Tạo chuỗi ngắn nhất chứa hai chuỗi

Giả sử muốn nối:

```
A = "abcde"  
B = "defgh"
```

Overlap:

```
"de"
```

Ta chỉ cần nối phần còn lại của **B**:

```
A + "fgh"
```

Kết quả:

```
"abcdefgh"
```

Code

```
string mergeStrings(string a, string b) {  
    int overlap = maximumOverlap(a, b);  
  
    return a + b.substr(overlap);  
}
```

32. Ứng dụng 12: Tìm số ký tự cần thêm để chuỗi trở thành chuỗi lặp hoàn chỉnh

Ví dụ:

```
s = "abcab"
```

Chu kỳ tiềm năng:

```
"abc"
```

Nếu thêm:

```
"c"
```

ta được:

```
"abcabc"
```

Công thức

```
period = n - pi[n - 1];
```

Số ký tự cần thêm:

```
(period - n % period) % period
```

Code

```
int minimumCharactersToAppend(string s) {  
    int n = s.size();  
  
    vector<int> pi = prefixFunction(s);  
  
    int period = n - pi[n - 1];  
  
    return (period - n % period) % period;  
}
```

33. Một cách hiểu trực quan về mảng pi

Xét:

```
s = "ababaca"
```

Mảng:

```
pi = 0 0 1 2 3 0 1
```

Giải thích:

```
pi[0] = 0  
"a"  
không có phần đầu và cuối giống nhau
```

```
pi[1] = 0  
"ab"  
không có
```

```
pi[2] = 1  
"aba"  
đầu "a" = cuối "a"
```

```
pi[3] = 2  
"abab"  
đầu "ab" = cuối "ab"
```

```
pi[4] = 3  
"ababa"  
đầu "aba" = cuối "aba"
```

```
pi[5] = 0  
"ababac"  
không có
```

```
pi[6] = 1  
"ababaca"  
đầu "a" = cuối "a"
```

34. Những lỗi rất hay gặp

Lỗi 1: Viết $j = \text{pi}[j]$

Sai:

```
j = pi[j];
```

Đúng:

```
j = pi[j - 1];
```

Vì prefix đang xét có độ dài j , nên vị trí cuối của nó là:

```
j - 1
```

Lỗi 2: Dùng if thay cho while

Sai:

```
if(j > 0 && s[i] != s[j]) {  
    j = pi[j - 1];  
}
```

Đúng:

```
while(j > 0 && s[i] != s[j]) {  
    j = pi[j - 1];  
}
```

Vì có thể cần lùi nhiều lần.

Lỗi 3: Quên tăng j

```
if(s[i] == s[j]) {  
    j++;  
}
```

Nếu quên dòng này thì `pi` luôn sai.

Lỗi 4: Quên đưa j về border sau khi tìm thấy pattern

```
if(j == m) {  
    positions.push_back(i - m + 1);  
  
    j = pi[j - 1];  
}
```

Nếu đưa:

```
j = 0;
```

vẫn có thể tìm được nhưng sẽ mất các lần xuất hiện chồng lấn.

Lỗi 5: Truy cập pattern[j] khi pattern rỗng

Thông thường đề bài đảm bảo:


```
m >= 1
```

Nếu pattern có thể rỗng thì phải xử lý riêng.

Lỗi 6: Chọn ký tự phân cách bị xuất hiện trong chuỗi

Nếu dùng:

```
pattern + "#" + text
```

thì cần đảm bảo:

```
'#'
```

không xuất hiện trong pattern và text.

35. Template KMP nên nhớ

```
vector<int> prefixFunction(string s) {  
    int n = s.size();  
  
    vector<int> pi(n);  
  
    for(int i = 1; i < n; i++) {  
        int j = pi[i - 1];  
  
        while(j > 0 && s[i] != s[j]) {  
            j = pi[j - 1];  
        }  
  
        if(s[i] == s[j]) {  
            j++;  
        }  
  
        pi[i] = j;  
    }  
  
    return pi;  
}
```

Đây là đoạn quan trọng nhất cần thuộc.

36. Template tìm pattern trong text

```
vector<int> KMP(string text, string pattern) {  
    vector<int> pi = prefixFunction(pattern);  
  
    vector<int> positions;  
  
    int j = 0;  
  
    for(int i = 0; i < (int)text.size(); i++) {  
        while(j > 0 && text[i] != pattern[j]) {  
            j = pi[j - 1];  
        }  
  
        if(text[i] == pattern[j]) {  
            j++;  
        }  
  
        if(j == (int)pattern.size()) {  
            positions.push_back(i - pattern.size() + 1);  
  
            j = pi[j - 1];  
        }  
    }  
  
    return positions;  
}
```

37. Khi nào nên nghĩ tới KMP?

Nếu đề có các từ khóa:

Tìm pattern trong text

Đếm số lần xuất hiện của chuỗi

Prefix bằng suffix

Border

Chu kỳ nhỏ nhất

Chuỗi được tạo bằng cách lặp

Prefix xuất hiện bao nhiêu lần

Prefix dài nhất xuất hiện K lần

Ghép chuỗi với độ chồng lấn lớn nhất

thì nên nghĩ tới:

KMP hoặc prefix function

38. Phân biệt KMP và Hash

KMP:

Ưu điểm:

- $O(n + m)$
- Không bị collision
- Rất mạnh với prefix, suffix, border, chu kỳ

Nhược điểm:

- Khó hiểu hơn lúc mới học
- Không linh hoạt bằng hash cho nhiều loại đoạn con

Hash:

Ưu điểm:

- So sánh hai substring nhanh
- Linh hoạt
- Dễ dùng với nhiều truy vấn

Nhược điểm:

- Có khả năng collision
- Cần xử lý modulo hoặc unsigned long long

Nếu bài tập trung vào:

```
prefix
suffix
border
period
pattern matching
```

thì KMP thường tự nhiên hơn.

39. Phân biệt pi và Z-function

Prefix function:

```
pi[i] = độ dài prefix dài nhất
        bằng suffix của đoạn s[0...i]
```

Z-function:

```
z[i] = độ dài đoạn bắt đầu tại i
        giống prefix của toàn bộ chuỗi
```

Ví dụ:

```
s = "ababab"
```

Prefix function:

```
pi = 0 0 1 2 3 4
```

Z-function:

```
z = 0 0 4 0 2 0
```

Hai thuật toán có nhiều ứng dụng giống nhau.

40. Tổng kết

KMP có hai phần chính:

Phần 1

Xây dựng prefix function:

```
pi[i]
```

Ý nghĩa:

Độ dài prefix dài nhất bằng suffix
trong đoạn $s[0...i]$

Phần 2

Khi gặp sai khác:

```
j = pi[j - 1];
```

Ta không quay về đầu mà tận dụng phần đã khớp.

Độ phức tạp:

```
 $O(n + m)$ 
```

Các ứng dụng quan trọng:

1. Tìm tất cả vị trí xuất hiện của pattern
2. Đếm số lần pattern xuất hiện
3. Tìm chu kỳ nhỏ nhất
4. Kiểm tra chuỗi có được tạo bằng cách lặp
5. Tìm border dài nhất
6. Liệt kê tất cả border
7. Đếm số lần mỗi prefix xuất hiện
8. Tìm prefix dài nhất xuất hiện ít nhất K lần
9. Prefix vừa là suffix vừa xuất hiện ở giữa
10. Tìm độ chồng lấn lớn nhất giữa hai chuỗi
11. Ghép hai chuỗi ngắn nhất
12. Tìm số ký tự cần thêm để hoàn thành chu kỳ

Câu quan trọng nhất cần nhớ:

Khi đang khớp được j ký tự nhưng gặp sai khác, ta thử border tốt nhất tiếp theo bằng $j = \text{pi}[j - 1]$, thay vì quay lại từ đầu.